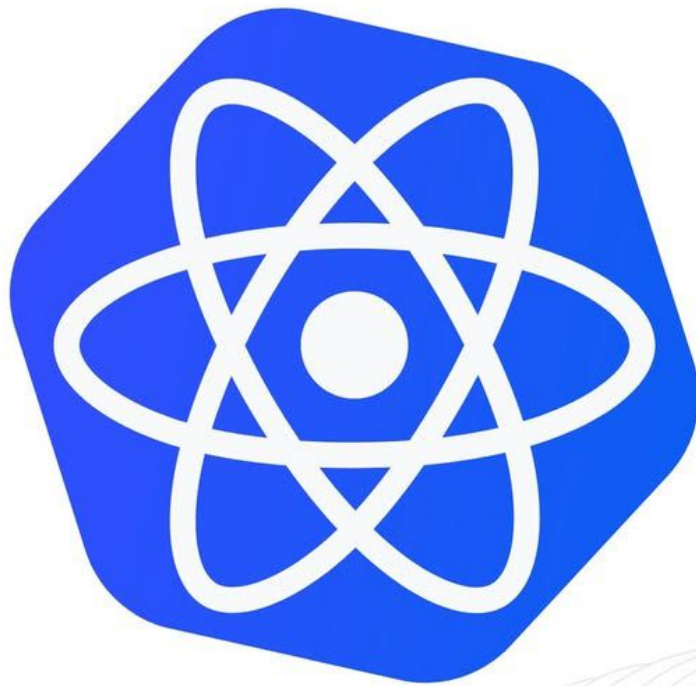


@JavaScriptsMagic

Proxy Setup In React

Avoid CORS Issues



What Is A Proxy In React?

- ◆ A Proxy In React Is Used To Avoid CORS Issues When Connecting Frontend To Backend During Development.
- ◆ It Forwards API Requests From Your React App To The Backend Server Seamlessly.

```
// Problem Without Proxy:  
fetch("http://localhost:5000/api/data")  
  
// Throws CORS Error If  
// Backend Not Configured Properly
```



Setting Up Proxy In package.json

- ◆ To Enable Proxying, Add The Proxy Key Inside Your React App's package.json File.
- ◆ This Tells React To Forward Requests Starting From "/" To The Backend URL.

```
// Problem Without Proxy:  
fetch("http://localhost:5000/api/data")  
  
// Throws CORS Error If  
// Backend Not Configured Properly
```



Using fetch With Proxy

- ◆ After Proxy Is Set, You Can Fetch Data Without Typing The Full Backend URL.
- ◆ Just Use The Relative Path And React Will Redirect It To The Proxy Server.

```
// Shortened URL Works Due To Proxy
fetch("/api/data")
  .then(res => res.json())
  .then(data => console.log(data));
```



Proxy With Different HTTP Methods

- ◆ Proxy Supports All HTTP Methods Like GET, POST, PUT, DELETE.
- ◆ You Can Use fetch Or Axios With Body And Headers Easily.

```
// Example POST Request
fetch("/api/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Amit" })
});
```



Proxy Setup With Multiple Environments

- ◆ For Complex Projects, Use http-proxy-middleware For Custom Rules.
- ◆ Helps Define Different Proxies For Development Vs Production.

```
// src/setupProxy.js
const { createProxyMiddleware } = require('http-proxy-middleware');

module.exports = function (app) {
  app.use("/api",
    createProxyMiddleware({ target: "http://localhost:5000",
    changeOrigin: true }));
};
```



Avoid Proxy In Production

- ◆ Proxy Setup In package.json Works Only In Development.
- ◆ In Production, React App And Backend Must Be Served From The Same Origin Or Use NGINX.
- ◆ You Can Also Handle CORS From The Backend Instead Of Using Proxy.

```
// Node.js Example For Handling CORS  
app.use(require("cors")());
```



Debugging Proxy Errors

- ◆ If Proxy Isn't Working, Ensure Backend Server Is Running & Routes Exist.
 - ◆ Restart React App After Adding Proxy To package.json.

Checklist:

- ☒ Is Backend Running?
- ☒ Are API Routes Correct?
- ☒ Is React App Restarted After Proxy Added?

```
# Restart React App After Adding Proxy  
npm start
```

